A Finite-State Checkers Player for Portable Systems

Christopher A. Tucker
cartheur@pm.me

Draft — November 2026

Abstract

This paper describes an unfinished deterministic Checkers player that can run on both a desktop and a small target system. Its controller is a finite-state machine (FSA) with named stages for receiving a board, checking it, generating moves, looking ahead, scoring moves, choosing one move, and reporting either a result or an error. The FSA controls the order of work; a separate rule ranks legal moves. The repository contains a portable C++11 core, a small mailbox protocol, board-geometry tests, and the first rules tests for uncrowned men. This draft explains the design and its tests. It does not claim strong play, complete Checkers rules, or a new result in game programming.

1 Introduction

Classic game systems show that a game player can be small and deterministic. This project asks whether such a player can be clear enough to inspect and test, while also running on a desktop emulator and small target hardware.

The answer proposed here is an FSA with named state, limited work per step, and visible mailbox results. Each stage can be tested on its own instead of hiding the whole process inside one search routine.

2 Architecture

The controller starts in q_0 = WAIT_REQUEST. It reads and checks a board, checks the result of the previous turn, and generates legal moves. It then enters q_4 = ANTICIPATE to project candidate lines before scoring them. A chosen move enters q_7 = DECISION; the next transition writes the response. Bad input or a terminal position enters q_E = ERROR; the next transition writes an error response. The source repository contains the state diagram and the formal input and output alphabets.

The FSA design is explicitly based on the treatment of finite-automaton behavior and synthesis by Trakhtenbrot and Barzdin [1]. The book supplies the starting discipline: state, stored information, inputs, outputs, and transitions must all be stated. This project is not presented as a new theorem in automata theory.

Current evidence artifact: `research/checkers-evidence/FSA-LOGIC.md`. The rendered diagram is `FSA-LOGIC.png`. At this draft stage the portable core implements the request/response skeleton; move generation provides evidence for the next generation state.

2.1 Separation of control and policy

The FSA is not the game strategy. It says which step comes next, what data is kept, and how errors are reported. `ANTICIPATE` is a state; projection is the work done in that state. `DECISION` and `ERROR` are also states; their outgoing transitions write to the mailbox. The strategy is a separate deterministic rule for ranking legal projected moves. It can change without changing the protocol or the rules checks.

3 Anticipation and decision

The key distinction is between looking at the consequences of a move and choosing the move to send to the game surface. Keeping those jobs separate makes a small player easier to test and explain.

3.1 `ANTICIPATE`: bounded projection, not commitment

The `ANTICIPATE` state takes one legal candidate and builds a private board position for it. At first, this means applying the move and recording captures, promotion, whose turn it is, and any required jump continuation. Later it may also look at a limited opponent reply and one reply after that. It does not predict what an opponent will do. It simply builds the positions that follow from a stated candidate line.

Three rules keep this work clear and small:

1. **Start with a legal move.** This state uses only moves produced by the rules engine.
2. **Keep the work private and limited.** The accepted board does not change. A scratch board and a trace are used instead. One FSA step does only a small unit of work, such as applying one move or one continuation.
3. **Keep the trace.** The trace records the candidate, projected board, captures, continuations, and later score. It shows why a move could be chosen and helps check the next board that arrives.

The state loops until it has made every projection requested for this turn. It does not stop merely because it has found a move that looks good. This gives the ranking rule a known set of inputs and keeps look-ahead from taking over the bus or the frame time.

3.2 `DECISION`: one move is made final

`DECISION` begins after every candidate has a score and `SELECT` has chosen one using a fixed ordering. It does not score moves again. It holds the chosen move and its trace until the result is written to the mailbox.

The transition out of `DECISION` writes the move data and sets `READY` last. A reader therefore cannot see a ready result before the move is complete. Each reported move can be traced back to its request, a legal candidate, its projected line, and the comparison that chose it.

Tie-breaking also belongs here. If two candidates have the same value, a fixed ordering—for example origin square followed by landing order—chooses one without randomness. The same input then gives the same result on the host and on the target, which helps testing and replay.

3.3 Why the separation matters

Without this split, a game program can mix up three questions: is a move legal, what follows from it, and should it be chosen? The FSA gives these to `GENERATE`, `ANTICIPATE`, and `SELECT/DECISION`. This does not guarantee strong play. It does make the source of a choice testable. A better ranking rule can be added without changing the rules, the limits on projection, or the mailbox order.

3.4 Portable boundary

The shared core is C++11 and has no target-specific dependencies. The host emulator and target firmware sit around the same board, mailbox, and agent interfaces. Hardware measurements are not yet available.

4 Validation methodology

Validation proceeds in layers:

1. Verify the 32-square board mapping, coordinate round trips, edge behavior, and invalid encodings with independent geometry fixtures.
2. Validate mailbox request parsing, sequencing, and error responses.
3. Fixture-test legal move generation and move application.
4. Add deterministic policy tests over fixed board positions.
5. Evaluate complete players against declared baselines using a documented, reproducible protocol.

Only the first three layers are partially implemented in this draft. The results section will remain intentionally incomplete until later layers and their measurements exist.

5 Current scope and limitations

The current rules implementation covers uncrowned-men quiet moves and single captures with compulsory-capture filtering. Promotion, king movement, multi-jumps, draw rules, policy, deeper projection, target adapters, and performance experiments remain future work. These limits are deliberate boundaries, not claims of completed functionality.

6 Related work and historical context

The present architectural basis is the behavioral and synthesis treatment of finite automata by Trakhtenbrot and Barzdin [1]. This section will grow into a reviewed comparison with Checkers programs, finite-state game controllers, constrained game hardware, and period computer-player implementations. The project has an initial lead list, but no novelty claim should be inferred before those sources are verified.

7 Evaluation plan

The completed paper will report:

- rules conformance against a documented position suite;
- determinism and protocol-error behavior;
- move quality against explicitly named baseline policies;
- memory, code size, and latency on each supported platform; and
- limitations, failure cases, and reproducibility instructions.

8 Conclusion

The project separates control, rules, and move ranking for a portable deterministic Checkers player. This draft records the design and the evidence available so far. Stronger claims must wait for the remaining rules, experiments, and related-work review.

References

[1] B. A. Trakhtenbrot and Ya. M. Barzdin. *Finite Automata: Behavior and Synthesis*. North-Holland Publishing Company, 1973. Translated from the Russian by D. Louvish; English translation edited by E. Shamir and L. H. Landweber.